Abstract: 243/250 words

Manuscript: 2724/4000 words

Preventing Silent Errors in Clinical Data Analysis Before the Code Runs: Design and Evaluation of a Novel Data Analysis Framework

Abstract

Objective

Mistakes in clinical and translational research data analytics are common, often challenging to identify, and lead to inaccurate outputs. Developing methods to minimize these errors is an established priority for clinical research. We designed and evaluated *Tidy-TS*, a TypeScript data analysis framework, to address this gap.

Materials and Methods

Tidy-TS is designed to preemptively identify potential coding errors by closely tracking how data types evolve throughout an analysis. Using 65 reproducible error scenarios across five categories, we evaluated how these errors are addressed in *Tidy-TS*, Python (*pandas*) and R (*tidyverse*) workflows. We tested *Tidy-TS* against an independent collection of 79 real-world data analysis bugs.

Results

Tidy-TS caught 62 of 65 analytical errors before code execution and immediately halted execution for the remaining 3 errors. Conversely, pandas and tidyverse silently executed 39 (60%) and 32 (49.2%) of error scenarios overall, and permitted silent errors in 100% and 88% of scenarios involving missing values, respectively. *Tidy-TS* detected all 79 real-world bugs from the external collection.

Discussion

This analysis demonstrates how data type errors in conventional clinical research workflows can lead to incorrect and often silent outputs. It highlights how type-safe data analysis can track data types and prevent an important class of errors.

Conclusion

Tidy-TS is a first-of-its-kind, open-source data analysis framework that prevents a pervasive class of silent errors in clinical data analysis before code execution. *Tidy-TS* offers a guardrail that directly addresses a well-documented source of error in clinical and translational research.

Background and Significance

Whether for research studies, quality improvement, or clinical operations, clinical data analysis involves many individual data processing steps. A standard workflow might query patient data

from a database or import it from a CSV file, reorganize it, combine it with data from other sources, calculate new values, summarize the results, and run statistical tests.

While these steps are routine, they are also error-prone. Some errors may be small and benign such as a typo when spelling the name of a CSV file. Others can have consequences large enough to change study conclusions. One review of 884 retracted articles in clinical and translational research found that 28% described problems preparing or analyzing data.[1] Minimizing these types of errors has been identified as a priority for clinical research.

Errors in analytical code can manifest in one of several ways. Some errors lead the code to halt as soon as the error is encountered. Some are more insidious, silently allowing code to run fully but corrupting results in ways that can be nuanced and hard to detect. These silent errors commonly involve issues related to the underlying type of data that code is trying to access or change. Non-technical researchers might be surprised to find that it is altogether an easy mistake to accidentally multiply a word by a number or combine two datasets in a way that silently drops hundreds of patients from a study.

Although these mistakes pose serious risks to clinical research, they stem from a broader challenge found across all of software development. This challenge is accelerating as analysts increasingly rely on AI-assisted programming tools, as AI-generated code often contains subtle logical flaws. To minimize these errors the software industry has introduced safeguards like automated test suites, strict validation frameworks, and peer reviews. While these development practices can significantly reduce bugs, they cannot fully eliminate risks that are inherent to the design of the programming languages themselves.

Most clinical data analysis is performed in dynamically typed languages like Python or R. Both languages can surface errors during code execution, but these checks typically only confirm whether a given line of code can technically run with some data. There is no inherent mechanism for broadly enforcing that the data accessed when the code runs actually aligns with the data an analyst expected throughout an analytical pipeline. For example, in Python, the numeric value `1` and the boolean value “True” can be used interchangeably in many operations. A string of characters, such as “MISSING”, can technically be multiplied by 2 and results in a value “MISSINGMISSING”. As a result, logically incorrect operations may still execute successfully without warning and lead to downstream incorrect results.

In contrast, statically typed programming languages are designed to catch these kinds of silent errors before the code runs. They use a compiler to keep track of what kind of data each variable is expected to contain. Prior to running the code, the compiler can pre-emptively detect if there are operations that do not align with the expected data types. It does this by monitoring how operations, throughout a program, may change the data type or introduce missing values. This kind of pre-execution type check is known as compile-time checking, and its theoretical foundation is well established.[2][3] Empirical studies of static type systems in JavaScript and Python have shown that 11–18% of real-world bugs would have been caught at compile time.[4][5]

Catching errors before any analytical code runs provides additional safeguards that are particularly relevant to clinical data processing. For complex research workflows, discovering an error hours into a computationally expensive analysis can waste valuable computational

resources and delays results. Worse, if the error does not halt execution, the script may output a seemingly accurate output with subtle incorrect results. Furthermore, as analytical and predictive models are increasingly deployed directly into production through clinical decision support tools, the consequences of runtime errors can scale. An unexpected crash or a silent error might result in a partially updated database or an incorrect clinical alert being triggered before the issue is caught.

TypeScript is a programming language with compile-time type checking that is already used in clinical informatics for health data standards, web dashboards, and data services.[6][7] Despite this adoption, TypeScript has not traditionally been used as a primary language for data analysis itself. Extending TypeScript to support data analysis offers a promising mechanism to reduce certain categories of analytical errors and support increasing regulatory expectations for formal evaluation of data quality in clinical software.[8][9].

We therefore evaluated whether *Tidy-TS,* a novel data analysis framework built in TypeScript, can detect common clinical data processing errors that would otherwise remain undetected in conventional Python and R workflows, and we characterize the practical limits of this approach.

Objective

The evaluation of *Tidy-TS* addressed three primary questions:

1. Can TypeScript's compile-time checking accurately track column names, data types, and missing values throughout standard clinical data processing steps to catch errors before code execution?
2. How does error detection in Tidy-TS compare to conventional analytical workflows in Python (pandas) and R (tidyverse) when exposed to equivalent clinical data errors?
3. What are the practical boundaries of compile-time checking, and under what conditions does runtime validation remain necessary?

Materials and Methods

System Design

Tidy-TS is a TypeScript framework for working with datasets. It represents the dataset as a *dataframe*, with data structured as a table with a collection of rows and named columns. Each column contains a specific type of data – numeric, boolean, text, etc. – that is strictly tracked by *Tidy-TS*. For the purposes of this study, the terms *dataset* and *dataframe* will be used interchangeably.

Compile-Time Checking

When a dataset is manipulated or edited, *Tidy-TS* tracks how the type of data contained in columns may change. When joining datasets – such as patient demographics and patient lab results – it is possible that some patients only appear in one of the two datasets. *Tidy-TS* keeps track of this possibility, marking data values as possibly missing and requiring that code address potentially missing values explicitly.

Each common dataset operation follows defined rules that determine how the column data types may change. The full set of type rules covering all operations is provided in *Supplementary Table 1.*

When importing external data, such as from a database or from files like CSV, XLSX, or Parquet, *Tidy-TS* ensures that any imported data matches the expected data types prior to proceeding with built-in data validation checks.

System Validation

Tidy-TS supports an array of common dataset operations, such as filtering, summarizing, modifying columns, and replacing missing values. It also provides functionalities to calculate descriptive statistics, hypothesis testing, and regression modeling. The accuracy of these operations, both related to tracking data types and the quantitative outputs, is validated by a suite of over 1,700 separate, reproducible, publicly-accessible tests. Functionalities for hypothesis testing and regression modeling are validated directly against R, with outputs aligning within a tolerance of 10^{-6}.

Figure 1 illustrates features of a data analysis workflow in *Tidy-TS*.

Language Comparison Suite

To compare error detection across *Tidy-TS*, Python, and R, we created a standalone, reproducible comparison suite of 65 common error scenarios for clinical data analysis. The test cases were derived from error patterns described in published clinical data quality literature[10][11][12][13][14] and from errors encountered during common data analysis tasks at the University of Utah.

Tests were organized into five common error categories for clinical data analysis: column reference errors, value type errors, missing value errors, dataset join errors, and dataset loading errors.

We implemented the error scenarios in *Tidy-TS, pandas* (3.0.3) with Python 3.14.5 and *tidyverse* (2.0.0) with R 4.6.0. *Tidy-TS* tests were run with TypeScript 5.9.2 on Deno 2.7.14, a runtime environment for TypeScript. The output of the error scenarios for each language were classified into one of four categories: compile-time error, error during execution, warning during execution, or silent continuation.

For completeness, the 65 error scenarios were also implemented in the Python *Polars (1.30.0)* data analysis library due to its increasing popularity, as well as with the *mypy and pyright* static type checking libraries intended to augment the base Python language and catch errors prior to code execution.

External Validation Against an Independent Bug Collection

To validate that the error scenarios suite translates to real-world bugs and does not selectively favor *Tidy-TS*, we evaluated *Tidy-TS* against a published collection of Python and R data analytics bugs.[15] The collection manually classified bugs from online forums and code repositories, labeling each bug by type, root cause, and effect. We selected the 164 bugs where the root cause of the failure was labeled as a Type Mismatch.

Among the Type Mismatch bugs, we excluded those with features that are inherently specific to a particular language or library and do not have a direct equivalent across programming languages. Each included bug is reproduced in a self-contained file that runs the original Python or R code and demonstrates the equivalent Tidy-TS implementation. For each bug, we also determined whether or not the original bug is reproducible in modern versions of Python and R libraries.

Results

Language Comparison Suite

The comparison suite included 65 tests across five error categories: 16 (25%) column reference errors, 14 (22%) value type errors, 17 (26%) missing value errors, 8 (12%) join errors, and 10 (15%) data loading errors. Each test was run in *Tidy-TS*, *pandas*, and *tidyverse*.

Tidy-TS caught 62 of 65 errors (95%) before code execution and intercepted the remaining 3 by halting execution when external data files were first loaded. *Pandas* halted execution for 26 (40%) and allowed silent continuation for 39 (60%). *Tidyverse* halted execution for 29 (45%), provided a text-based warning for 4 (6%), and allowed silent continuation for 32 (49%) (*Table 1, Figure 2*).

The Python static checking libraries *mypy* and *pyright* did not catch any of the 65 errors prior to code execution (*Supplementary Table 2*). The *Polars* data analysis library halted code execution for 28 of 65 errors (43%), performing comparably to *pandas* and *tidyverse*. Like pandas, *Polars* allowed silent continuation for all 17 missing value errors.

If pre-execution checks were explicitly bypassed, *Tidy-TS* halted code execution for 36 (55%) of the 65 errors.

The largest detection gap occurred with missing values, where pandas and tidyverse allowed 17 (100%) and 15 (88%) error scenarios to continue silently, respectively. In contrast, column reference errors, where a provided column name was misspelled or non-existent, showed the smallest detection gap, with *pandas* and *tidyverse* both halting execution for 14 of 16 (88%) scenarios.

External Validation Against an Independent Bug Collection

Of the 164 Type Mismatch bugs, 79 met the inclusion criteria of having a direct equivalent across programming languages applicable to general data analytics. Among the 79 included bugs, there were 60 (76%) data type errors, 9 (11%) missing value errors, 5 (6%) data loading errors, 4 (5%) column reference errors, and 1 (1%) dataset join error. All 79 bugs were caught by *Tidy-TS* before code execution.

Of the 79 included bugs, 62 (80%) still trigger the original failure in current versions of the respective Python and R libraries. The remaining 17 no longer reproduce the original silent failure, reflecting upstream changes in those libraries since the bugs were originally reported.

Discussion

Both our publicly-available comparison suite and an external collection of real-world bugs illustrate how a novel compile-time safe data analytics library catches numerous silent errors

that occur in two of the most commonly used programming languages for clinical data analysis. The gap was most prominent in cases where missing values might enter an analysis, potentially producing plausible but systematically incorrect output *(Figure 3*).

Python's optional static analysis tools did not address this limitation. Even with the most thorough settings enabled, *mypy* and *pyright* failed to detect any of the 65 errors. While these tools can describe the types of individual values, they do not integrate with pandas to track how dataset columns and missing values change throughout a data analysis. The increasingly popular *Polars* data analysis library for Python likewise did not meaningfully outperform *pandas* or *tidyverse*.

These findings are situated within a broader literature documenting the prevalence and consequences of analytical errors in clinical research. Broad analyses of thousands of retracted medical publications have found that data concerns were the single most common reason for retraction at 31.5%.[16] An earlier study of 395 retracted articles found that unintentional errors — mistakes in sampling, procedures, or data analysis — accounted for more than twice as many retractions as scientific misconduct.[17] A survey of 97 researchers whose papers were retracted due to unintentional errors found that inattention was the most frequently cited cause, and that errors could occur at any stage of the data management workflow.[18] Taken together, these findings suggest that a substantial share of retracted clinical research may stem from the kind of routine, silent processing errors that compile-time type systems are specifically designed to prevent.

As clinical research and informatics increasingly make use of AI-assisted programming, the risk of silent errors is magnified. AI outputs frequently hallucinate APIs and generate plausible but logically flawed code that languages like Python or R will execute without warning.[19] We anticipate that data analysis tools that incorporate pre-execution type checks like *Tidy-TS* can serve as an important guardrail to permit AI agents to author increasingly complex, error-free clinical data analyses. Future work will assess the real-world utility of *Tidy-TS* in this regard.

Tidy-TS occupies a different domain of error prevention than existing tools that primarily evaluate the completeness, conformance, or plausibility of existing clinical data repositories, such as the OHDSI Data Quality Dashboard.[20] Data handling during analysis has been identified as a distinct source of error that these approaches do not address.[21][22] *Tidy-TS* and compile-time data analytics help directly bridge this gap.

Runtime validation libraries in Python and R, such as *pandera* and *pointblank*, also target errors within the analysis workflow. However, these systems depend on analysts anticipating each possible failure condition and adding checks manually. *Tidy-TS* applies equivalent checks automatically through the compiler, whether or not the analyst anticipated the problem in advance. This distinction is most important for errors that are difficult to predict before the analysis is written.

Limitations

While compile-time programming languages can prevent the implementation of accidental analytical errors related to unexpected data types, there remain other analytical errors that may go undetected. For example, if an analyst mistakenly substitutes one date value for another –

perhaps referencing a medication order date rather than a medication start date – *Tidy-TS* cannot alone account for this error. Likewise, *Tidy-TS* cannot necessarily prevent an analyst from selecting an incorrect hypothesis test or incorrectly specifying a logistic regression. This limitation reflects a distinction between verifying analytical correctness and scientific validity. The latter still requires domain expertise regardless of the programming environment. However, there is immense versatility in type-safe programming languages, and the foundation that *Tidy-TS* provides could potentially be extended to help address an additional set of these errors.

Languages that provide pre-execution type checks are also sometimes perceived as adding friction to the software development process. In practice, many of these checks correspond to steps that will ultimately be required for a correct clinical analysis, regardless of programming language. Indeed, the discussed literature on the prevalence of errors in clinical research implies that analysts may be too quick to accept code that runs as correct code. In our experience, requiring that these issues be addressed prior to code execution helps uncover numerous assumptions and is ultimately time-saving.

As noted in the analysis, languages that provide pre-execution type checks alone cannot verify that external data actually aligns with the expected shape. This necessitates runtime validation, which Tidy-TS enforces as a strict requirement during data loading to safely support all downstream type checks.

TypeScript also has practical limits. Programmers can bypass the compiler, and workflows with particularly complex or poorly designed types can exceed compiler capacity. *Tidy-TS* has been optimized to minimize compiler burden and actively supports ongoing projects at the University of Utah using complex type systems.

Conclusion

Tidy-TS is a novel, open-source data analytics library that prevents a common class of errors in clinical data analysis before code is executed. It catches numerous errors that execute silently in Python and R workflows, with the largest detection gap related to potentially missing values introduced during analysis. In doing so, it represents a first-of-its-kind tool designed to directly address a well-documented source of error in clinical and translational research.

Acknowledgements

Multiple AI tools (Anthropic Claude, Google Gemini, OpenAI ChatGPT) were used throughout the project, to various extents, for library development and manuscript-related analyses. All scientific content, study design, analysis, interpretations, and manuscript decisions were made by the authors, who take full responsibility for the work.

Tidy-TS builds on ideas and infrastructure from several open-source projects, including R, Python, *pandas*, *Polars*, *tidyverse*, *arquero*, and *zod*. The authors are grateful to these communities for their work.

Data Availability

The Tidy-TS library, its extensive testing suite, the external validation bug collection, and the full comparison suite of 65 error scenarios (with equivalent implementations in Tidy-TS, pandas,

tidyverse, and Polars) are publicly available at https://github.com/jtmenchaca/tidy-ts (release 2.0.0). All outcomes reported in this paper can be reproduced from the published source code.

Funding Statement: The authors received no financial support for the research, authorship, and/or publication of this article.

Conflicts of Interest: The authors have no competing interests to declare.

References

1. Baldridge AS, Bellinger GC, Fleming OM, Rasmussen LV, Whitley EW, Welty LJ. The epidemiology of errors in data capture, management, and analysis: A scoping review of retracted articles and retraction notices in clinical and translational research. Journal of Clinical and Translational Science. 2024;8(1):e174. doi:10.1017/cts.2024.533.
2. Milner R. A Theory of Type Polymorphism in Programming. Journal of Computer and System Sciences. 1978;17(3):348-375. doi:10.1016/0022-0000(78)90014-4.
3. Cardelli L, Wegner P. On Understanding Types, Data Abstraction, and Polymorphism. ACM Computing Surveys. 1985;17(4):471-523. doi:10.1145/6041.6042.
5. Khan F, Chen B, Varro D, Khan S. An Empirical Study of Type-Related Defects in Python Projects. IEEE Transactions on Software Engineering. 2022;48(8):3145-3158. doi:10.1109/TSE.2021.3082068.
7. Gøeg KR, Rasmussen RK, Jensen L, Wollesen CM, Larsen S, Pape-Haugaard LB. A Future-Proof Architecture for Telemedicine Using Loose-Coupled Modules and HL7 FHIR. Computer Methods and Programs in Biomedicine. 2018;160:95-101. doi:10.1016/j.cmpb.2018.03.010.
8. Warraich HJ, Tazbaz T, Califf RM. FDA Perspective on the Regulation of Artificial Intelligence in Health Care and Biomedicine. JAMA. 2025;333(3):241-247. doi:10.1001/jama.2024.21451.
9. Labkoff S, Oladimeji B, Kannry J, et al. Toward a Responsible Future: Recommendations for AI-Enabled Clinical Decision Support. Journal of the American Medical Informatics Association : JAMIA. 2024;31(11):2730-2739. doi:10.1093/jamia/ocae209.
10. Razzaghi H, Wieand K, Dickinson KL, et al. Beyond Missingness: Systematizing Methods for Comprehensive Data Fitness Assessment in Clinical Research. Journal of Medical Internet Research. 2026;28:e76398. doi:10.2196/76398.
11. van Essen MHJ, Twickler R, Weesie YM, et al. Implications of Data Extraction and Processing of Electronic Health Records for Epidemiological Research: Observational Study. Journal of Medical Internet Research. 2025;27:e64628. doi:10.2196/64628.
12. Priou S, Kempf E, Jankovic M, Lamé G. “Goldmine” or “Big Mess”? An Interview Study on the Challenges of Designing, Operating, and Ensuring the Durability of Clinical Data Warehouses in France and Belgium. Journal of the American Medical Informatics Association : JAMIA. 2024;31(11):2699-2707. doi:10.1093/jamia/ocae244.
13. Denney MJ, Long DM, Armistead MG, Anderson JL, Conway BN. Validating the Extract, Transform, Load Process Used to Populate a Large Clinical Research Database. International Journal of Medical Informatics. 2016;94:271-4. doi:10.1016/j.ijmedinf.2016.07.009.
14. An D, Lim M, Lee S. Challenges for Data Quality in the Clinical Data Life Cycle: Systematic Review. Journal of Medical Internet Research. 2025;27:e60709. doi:10.2196/60709.
15. Ahmed S, Wardat M, Bagheri H, Dantas Cruz B, Rajan H. Characterizing Bugs in Python and R Data Analytics Programs. arXiv preprint. 2023. arXiv:2306.08632.
16. Okyay RA, Kocyigit BF, Qumar AB, Yessirkepov M, Sumbul HE. Fifty Years of Retracted Medical Publications From 1975 to 2024: A Comprehensive Analysis of Trends, Reasons, and Countries Using the Retraction Watch Database. Journal of Korean Medical Science. 2025;40(46):e300. doi:10.3346/jkms.2025.40.e300.
17. Nath SB, Marcus SC, Druss BG. Retractions in the Research Literature: Misconduct or Mistakes?. Medical Journal of Australia. 2006;185(3):152-4. doi:10.5694/j.1326-5377.2006.tb00504.x.
18. Kovacs M, Varga MA, Dianovics D, Poldrack RA, Aczel B. Opening the Black Box of Article Retractions: Exploring the Causes and Consequences of Data Management Errors. Royal Society Open Science. 2024;11(12):240844. doi:10.1098/rsos.240844.
19. Zhang Z, Wang Y, Wang C, Chen J, Zheng Z. LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation. Proceedings of the ACM on Software Engineering. 2025;2(FSE):Article 75. doi:10.1145/3728894.

20. Blacketer C, Defalco FJ, Ryan PB, Rijnbeek PR. Increasing Trust in Real-World Evidence Through Evaluation of Observational Data Quality. Journal of the American Medical Informatics Association : JAMIA. 2021;28(10):2251-2257. doi:10.1093/jamia/ocab132.
21. Sauer CM, Chen LC, Hyland SL, Girbes ARJ, Elbers PWG, Celi LA. Leveraging Electronic Health Records for Data Science: Common Pitfalls and How to Avoid Them. The Lancet Digital Health. 2022;4(12):e893-e898. doi:10.1016/S2589-7500(22)00154-6.
22. Blacketer C, Defalco FJ, Ryan PB, Rijnbeek PR. Increasing Trust in Real-World Evidence Through Evaluation of Observational Data Quality. Journal of the American Medical Informatics Association : JAMIA. 2021;28(10):2251-2257. doi:10.1093/jamia/ocab132.
23. Kohane IS, Aronow BJ, Avillach P, et al. What Every Reader Should Know About Studies Using Electronic Health Record Data but May Be Afraid to Ask. Journal of Medical Internet Research. 2021;23(3):e22219. doi:10.2196/22219.
24. Blacketer C, Defalco FJ, Ryan PB, Rijnbeek PR. Increasing Trust in Real-World Evidence Through Evaluation of Observational Data Quality. Journal of the American Medical Informatics Association : JAMIA. 2021;28(10):2251-2257. doi:10.1093/jamia/ocab132.
25. Sauer CM, Chen LC, Hyland SL, Girbes ARJ, Elbers PWG, Celi LA. Leveraging Electronic Health Records for Data Science: Common Pitfalls and How to Avoid Them. The Lancet Digital Health. 2022;4(12):e893-e898. doi:10.1016/S2589-7500(22)00154-6.

Figure 1. Architecture of a Type-Safe Clinical Data Workflow in *Tidy-TS*

To prevent silent errors, *Tidy-TS* validates external clinical data against an expected structure when it is first loaded. Once loaded, the compiler continuously tracks data types and potentially missing values across all subsequent transformations and statistical tests. This ensures analytical errors are caught before the code runs, safely supporting the generation of final operational and analytical outputs.

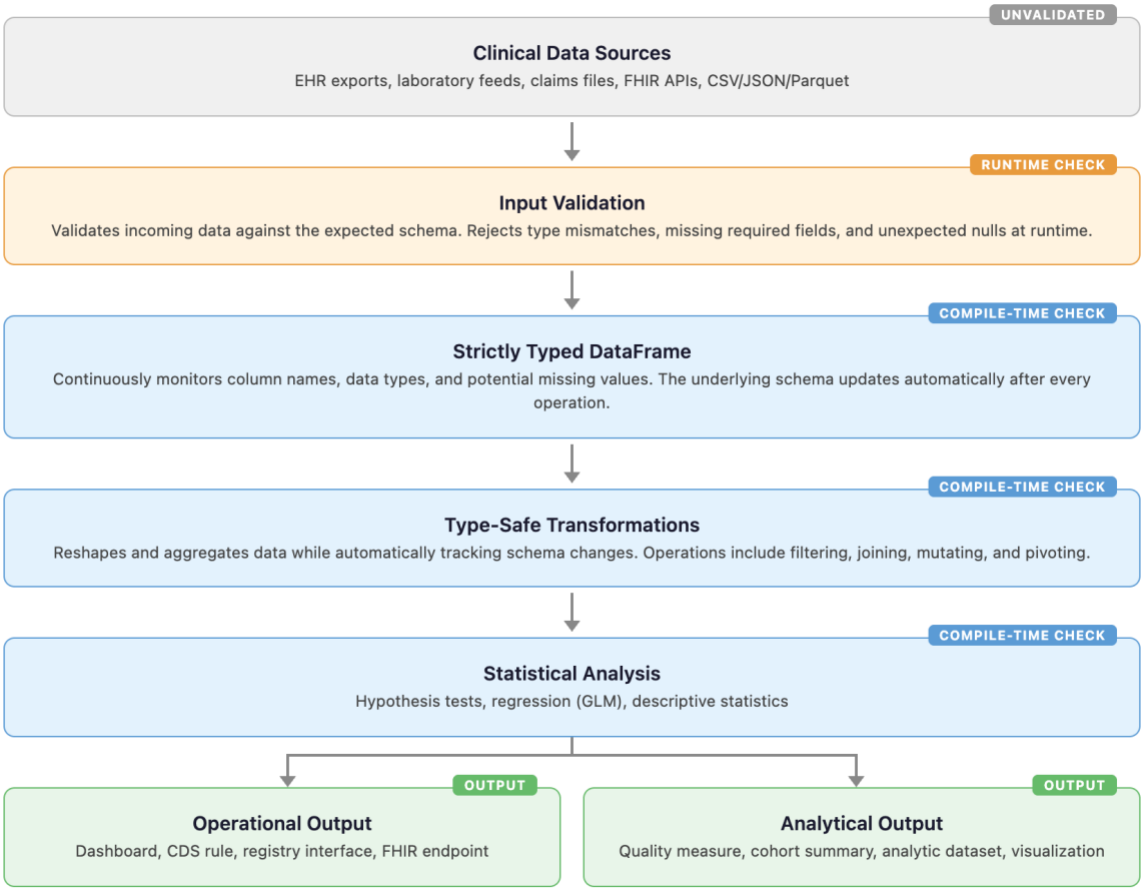

Figure 2. Error Detection Rates Across Analytical Frameworks

A heatmap comparing the proportion of simulated data analytics errors (n=65) caught prior to execution in *Tidy-TS*, Python (*pandas*), and R (*tidyverse*). An error is classified as "caught" if the framework immediately halted execution or generated an explicit text-based warning. Tidy-TS successfully intercepted 100% of errors across all categories, whereas conventional frameworks demonstrated a severe detection gap regarding null and missing data.

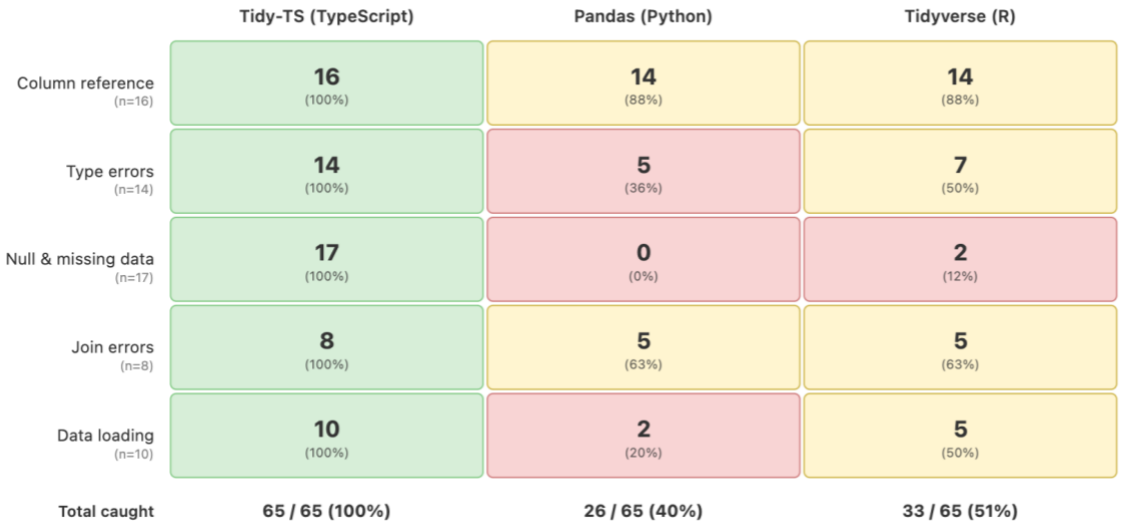

Figure 3. Example Comparison of Error Detection Between Frameworks.

When attempting to multiply a column containing a text value ("MISSING"), the frameworks exhibit three distinct behaviors. *Tidy-TS* detects the invalid operation and highlights the error before execution. *Pandas* executes silently, propagating a logical error by repeating the string ("MISSINGMISSING"). *Tidyverse* encounters a fatal error during execution and halts at runtime.

Table 1. Detection Outcomes by Error Category.

Detection outcomes across 65 simulated error scenarios. For *Tidy-TS*, the 'compile' column indicates errors successfully intercepted before execution, while the 'If bypassed' column indicates errors that would still trigger a runtime halt if static typing checks were explicitly overridden by the analyst.

			Tidy-TS (TypeScript)		**Pandas (Python)**			**Tidyverse (R)**		
			compile	*If bypassed*	*runtime*			*runtime*		
Category	**Example error**	**n**	*error*	*error*	*error*	*warn*	*silent*	*error*	*warn*	*silent*
Column reference	Use ‘patientId’ instead of ‘patient_id’	16	16	16	14	0	2	14	0	2
Value type errors	Including written text in an arithmetic operation	14	13	3	5	0	9	4	3	7
Missing value errors	Divide by a reference range value that may be missing	17	17	3	0	0	17	2	0	15
Join errors	Use a department column that may be missing after joining patients to encounters	8	8	6	5	0	3	5	0	3
Data loading	Load lab results expecting numeric data, but receiving a text entry like "pending"	10	8	8	2	0	8	4	1	5
Total		**65**	**62**	**36**	**26**	**0**	**39**	**29**	**4**	**32**

Supplemental Table 1: Type Rules in Tidy-TS data frames

Let ρ denote a row type (i.e., a mapping from column names to types).

For a set of columns C:

$\rho[C]$ denotes projection onto C,

$\rho \setminus C$ denotes removal of columns in C,

$\rho \cup S$ denotes schema union, assuming disjoint column names or an explicitly specified collision-resolution policy

$T \mid \bot$ denotes an optional value of type T, where $\bot$ corresponds to an absent or undefined value.

Lowercase identifiers a, b, c, k, n, v, and t range over column names.

For joins, L and R denote the left and right input schemas, respectively, and K denotes the join key set.

Category	Operation	Type rule	Description
Schema preservation	`filter`	$\rho \to \rho$	Keep rows matching a predicate
Schema narrowing	`select(C)`	$\rho \to \rho[C]$	Keep only columns C
	`drop(C)`	$\rho \to \rho \setminus C$	Remove columns C
	`distinct(C)`	$\rho \to \rho[C]$	Deduplicate, keep only columns C
Schema extension	`mutate({k: fn}), fn: (r: ρ) => T`	$\rho \to \rho \cup \{k{:}\, T\}$	Add column k with type T
	`mutate({k: fn}), fn: (r: ρ) => T`	$\rho \to (\rho \setminus \{k\}) \cup \{k{:}\, T\}$	Replace column k's type with T
	`rename({a: b})`	$\rho \to (\rho \setminus \{a\}) \cup \{b{:}\ \rho[a]\}$	Renames column a to b, preserves its type
Schema replacement	`summarize(S)`	$\rho \to S$	Only summary columns remain
	`groupBy(G).summarise(S)`	$\rho \to \rho[G] \cup S$	Group keys + summary columns
Joins *(shared non-key columns get a suffix (default _x/_y)*	`innerJoin(L, R, K)`	$L \cup (R \setminus \mathrm{K})$	All fields required
	`leftJoin(L, R, K)`	$L \cup \{\forall f \in R \setminus \mathrm{K}{:}\, \mathrm{f} \mid \bot\}$	Right non-key fields may be undefined
	`rightJoin(L, R, K)`	$\{\forall f \in L \setminus \mathrm{K}{:}\, \mathrm{f} \mid \bot\} \cup R$	Left non-key fields may be undefined
	`outerJoin(L, R, K)`	$L[K] \cup \{\forall f \in L \setminus \mathrm{K}{:}\, \mathrm{f} \mid \bot\} \cup \{\forall f \in R \setminus \mathrm{K}{:}\, \mathrm{f} \mid \bot\}$	Both sides' non-key fields may be undefined
Missing value handling	`replaceNull({k: v}), v: T`	$\rho \to (\rho \setminus \{\mathrm{k}\}) \cup \{\ \mathrm{k}{:}\ (\rho[\mathrm{k}] \setminus \mathrm{null}) \cup \mathrm{T}\ \}$	Substitute v for null; null is removed but T joins k's type
	`replaceUndefined({k: v}), v: T`	$\rho \to (\rho \setminus \{\mathrm{k}\}) \cup \{\ \mathrm{k}{:}\ (\rho[\mathrm{k}] \setminus \bot) \cup \mathrm{T}\ \}$	Substitute v for undefined; ⊥ is removed but T joins k's type
	`removeNull(k)`	$\rho \to (\rho \setminus \{\mathrm{k}\}) \cup \{\ \mathrm{k}{:}\ (\rho[\mathrm{k}] \setminus \mathrm{null})\ \}$	Drop rows where k is null, narrow k's type
	`removeUndefined(k)`	$\rho \to (\rho \setminus \{\mathrm{k}\}) \cup \{\ \mathrm{k}{:}\ (\rho[\mathrm{k}] \setminus \bot)\ \}$	Drop rows where k is undefined, narrow k's type

Supplemental Table 1. Detection Outcomes by Error Category, including Python Static Type Checker Coverage and Polars.

Detection outcomes for *pyright (1.1.400)* and mypy (1.16.0) running in strict mode on the same 65 error scenarios, with *pandas-stubs (2.2.3.241126)* installed. *Polars (1.30.0)* runtime results are included for comparison.

		Tidy-TS		**pyright**	**mypy**	**Polars**	**pandas**	**tidyverse**
Category	**n**	*compile*	*If bypassed*	*compile*	*compile*	*runtime*	*runtime*	*runtime*
Column reference	16	16 (100%)	16 (100%)	0 (0%)	0 (0%)	14 (88%)	14 (88%)	14 (88%)
Value type	14	13 (93%)	3 (21%)	0 (0%)	1 (7%)	6 (43%)	5 (36%)	4 (29%)
Missing value	17	17 (100%)	3 (18%)	0 (0%)	0 (0%)	0 (0%)	0 (0%)	2 (12%)
Join errors	8	8 (100%)	6 (75%)	0 (0%)	0 (0%)	3 (38%)	5 (63%)	5 (63%)
Data loading	10	8 (80%)	8 (80%)	0 (0%)	0 (0%)	5 (50%)	2 (20%)	5 (50%)
Total	65	62 (95%)	36 (55%)	0 (0%)	1 (2%)	28 (43%)	26 (40%)	29 (45%)